

A Preliminary Study on Sensitive Information Exposure Through Logging

Chen Zhi, Jianwei Yin, Junxiao Han, Shuiguang Deng
Zhejiang University, Hangzhou, China
{zjuzhichen, zjuyjw, junxiaohan, dengsg}@zju.edu.cn

Abstract—Logging is a common practice to collect valuable runtime information about software systems. However, information written to log files can be sensitive and give valuable guidance to attackers. In fact, information exposure through logging is not uncommon. Even large-scale online services (e.g., Facebook and Twitter) have reported exposing sensitive information via log files, and hundreds of millions of users are affected. Despite the severity of such vulnerabilities, there is no existing work that studies such vulnerabilities in the real-world context, and we have little knowledge about them. To fill this gap, we conduct a preliminary study on 413 real-world vulnerabilities to investigate the exploitability and root causes of such vulnerabilities. By analyzing these vulnerabilities, we find that 1) about two-third (67.8%) vulnerabilities can be exploited via the network, and a significant amount (89.3%) of vulnerabilities can be exploited with low efforts; 2) malicious users and insiders can use about half of (46.9%) proof-of-concept exploits to launch attacks without any expertise; 3) the top three common root causes for the vulnerabilities are insecure whole-object logging (43.4%), incorrect permission assignment (17.5%), and improper implementation of sanitization (11.2%). Based on the findings, we also discuss the implications for researchers and practitioners. We believe our work can inspire further work on detecting and fixing the vulnerabilities.

Keywords—Information exposure, information leakage, logging, vulnerabilities

I. INTRODUCTION

Information exposure (or leakage) is an emerging security threat to companies and organizations. Specifically, in the 2019 Common Weakness Enumeration (CWE) Top Most Dangerous Software Errors list [1], information exposure (CWE-200)¹ take fourth place among all 1,000+ common weakness. It indicates that information exposure is one of the most frequent and critical errors that can lead to serious software vulnerabilities.

Information exposure can occur in different ways and any place. Logging is an important form of exposing sensitive information. In the 2019 CWE Most Dangerous Software Errors list, information exposure through logging (CWE-532)² is in the 35th place. It is well-known that logging is a common practice to record runtime information for realtime analysis and postmortem inspection. However, the prevalence of logging also increases the potential risk of sensitive information exposure. The security posture of the logging system usually is much laxer than other storage systems (such as the database). It is not uncommon that log files are stored

at the client/server's file system with weak protection. Even though prevention solutions are introduced to protect sensitive information, it is also possible to write confidential data to log files inadvertently. For example, to avoid direct use of the actual password, Facebook utilizes hashing functions to replace the actual password with a random set of characters. However, the hashing is skipped under certain conditions, which results in original passwords being stored in log files and searchable by Facebook employees. Hundreds of millions of Facebook (including Instagram) users are affected by this vulnerability [2]. In the rest of this paper, we use the term *vulnerability* to refer to security vulnerabilities exposing sensitive information through logging.

Despite the severity of information exposure through logging, there is no existing work that studies such vulnerabilities in the real-world. In particular, previous research mainly focus on designing new approaches to detect these vulnerabilities (especially for Android applications [3]) instead of characterizing confirmed real-world vulnerabilities. It is still necessary to understand the nature of such vulnerabilities so that we can devise practical solutions to combat them. In this paper, we perform a preliminary study to answer the following two research questions to fill this gap.

RQ1: How exploitable are the vulnerabilities? This research question can give us hints on how attackers can exploit the vulnerabilities.

RQ2: What are the root causes of the vulnerabilities? This research question is helpful in shedding light on how to detect and repair the vulnerabilities.

This paper makes the following main contributions:

- To the best of our knowledge, our work is the first dedicated to exploring the real-world vulnerabilities of sensitive information exposure through logging.
- Based on analyzing the study results, we reveal three interesting findings for the exploitability and root causes of vulnerabilities. Furthermore, we discuss the implications from the perspectives of researchers and practitioners, which can inspire further work on detecting and fixing the vulnerabilities.

The remainder of this paper is organized as follows. Section II presents the setup of our empirical study. Section III presents the result of our empirical study. Section IV discusses the implications and limitations of our study. Section V presents the related works. Section VI concludes and outlines future work.

¹<https://cwe.mitre.org/data/definitions/200.html>

²<https://cwe.mitre.org/data/definitions/532.html>

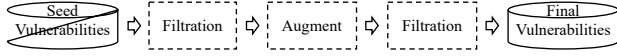


Fig. 1: Process of Vulnerabilities Collection

II. STUDY SETUP

This section narrates our study setup in detail. We first introduce the process to collect vulnerabilities, and then describe how we extract information to answer our research questions.

A. Vulnerabilities Collection

The vulnerabilities are collected from the National Vulnerability Database (NVD), which is one of the leading public vulnerability databases. We download entire database via the NVD JSON Vulnerability Feeds at 2020-05-30. After that, we follow the steps showed in Figure 1 to construct our dataset.

We first collect all vulnerabilities labeled with CWE-532 as the seed vulnerabilities, because it is the most relevant category for information exposure through logging. However, due to the ambiguity and overlap between categories, there are some misclassified vulnerabilities. On the one hand, some irrelevant vulnerabilities are labeled with CWE-532. To solve this problem, we manually examine the descriptions of vulnerabilities to remove such vulnerabilities. On the other hand, some relevant vulnerabilities are not labeled with CWE-532. It is necessary to augment our dataset by adding these vulnerabilities. To achieve this, we write a simple script to perform the keyword-based search on all vulnerabilities. The keywords³ are derived from the descriptions of those valid vulnerabilities labeled with CWE-532. After that, we manually examine the descriptions of vulnerabilities to discard irrelevant vulnerabilities. Besides, we also perform time-based filtration³ to remove old vulnerabilities. Because there are insufficient resources to understand these vulnerabilities, and we can gain little insight from them. By doing so, we get the final dataset with **413** valid vulnerabilities.

B. Exploitability of vulnerabilities

To answer RQ1, we use the Common Vulnerability Scoring System (CVSS) to measure the exploitability of vulnerabilities. CVSS is one of the most widely-used systems for assessing vulnerabilities, and many vulnerabilities databases provide CVSS scores. For example, the security analysts in NVD calculate the CVSS scores based on the details of the vulnerabilities and existing CVSS scores provided by developers. As Johnson et al. [4] figured out, the CVSS scores on NVD are relatively more consistent and trustworthy compared with other vulnerabilities databases. Therefore, we use the CVSS scores from NVD to answer RQ1. To be specific, we adopt two metrics in CVSS, as listed below:

- **Attack Vector** reflects the paths or means by which attackers can gain access to information or systems. $AV \in \{Physical, Local, Adjacent\ Network, Network\}$.

³The details can be found in our supplementary materials. <https://github.com/coder-chenzhi/LogLeak>

TABLE I: Top 10 Open Source Software Projects

Software Projects	Short Description	Main Lang.	Freq.
Cloud Foundry	PaaS platform	Go	12
Jenkins	CI/CD system	Java	12
oVirt	Virtualization platform	Python	10
Ansible	IT automation platform	Python	9
OpenStack	IaaS platform	Python	8
Logstash	Log processing system	Ruby	4
Aquarius CMS	Content management system	PHP	3
GitLab	Repository management system	Ruby	3
Moodle	Learning management system	PHP	3
OpenShift	Container management platform	Go	3
Subtotal	N/A	N/A	67
Other	N/A	N/A	76
Total	N/A	N/A	143

- **Attack Complexity** measures how complex are the prerequisite beyond the attacker's control for exploits. $AC \in \{Low, High\}$.

To enable a better understanding of the exploitability, we also investigate the PoC (proof of concept) exploits of vulnerabilities, which are usually provided by security researchers to demonstrate vulnerabilities. Moreover, PoC exploits can guide attackers to create real-world exploits. To collect PoC exploits, we examine the reference resources tagged with "Exploit" for each vulnerability in NVD. By doing so, we have collected the PoC exploits for **49** vulnerabilities. Most of these PoC exploits are from PoC databases, issue tracking systems, and security researchers' blogs.

C. Root cause of vulnerabilities

There are often insufficient resources to determine the root causes of the vulnerabilities in closed-source software/firmware/hardware. Therefore, we focus on the vulnerabilities in open-source software (OSS) projects. To achieve this, we need to identify the affected OSS projects for each vulnerability. We first use the Common Platform Enumeration (CPE)⁴ identifiers on NVD to extract the affected products. CPE is a standardized method of describing classes of software applications, operating systems, and hardware devices. After that³, we search the source code for the affected products on the Internet to determine whether it is OSS projects. It turns out that there are **143** vulnerabilities affecting OSS projects. Table I shows the top 10 OSS projects by the number of associated vulnerabilities.

To determine the root causes of the vulnerabilities in OSS projects, we follow the card sorting [5] approach to categorize these vulnerabilities by examining their associated information (description, advisories, patches, issue reports, etc.). Our card sort is open, which means there are no predefined groups. The detailed process consists of two phases: 1) in the *preparation* phase, we create one card for each vulnerability along with a brief description about how the vulnerability manifests; 2) in the *execution* phase, we sort the cards into groups with an self-explanatory title. The process is performed by the first and third authors independently. After completing the manual sorting process, the two sorters discussed their disagreements to reach a common decision.

⁴<https://nvd.nist.gov/products/cpe>

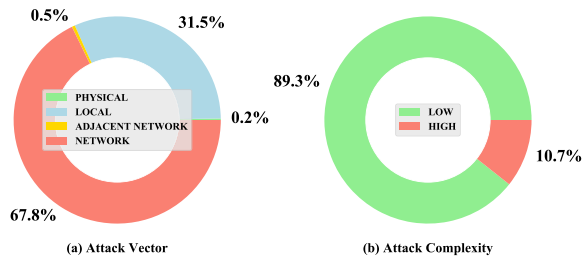


Fig. 2: Distribution of Exploitability Metrics

III. RESULT

A. RQ1: How exploitable are the vulnerabilities?

1) *Analysis of CVSS scores:* As explained in Section II, we use two metrics in CVSS to measure the exploitability of vulnerabilities. Figure 2 shows the distribution of these metrics in our dataset.

Attack Vector (AV) reflects how the attackers can access to vulnerable systems for exploits. The more remote a vulnerability can be exploited, the more potential attackers can exploit the vulnerability, the more severe the vulnerability is. As shown in Figure 2(a), there are about two-third (67.8%, 280/413) vulnerabilities can be exploited across the network, even up to the entire Internet. Most of the rest vulnerabilities (31.5%, 130/413) need to be exploited locally, which means the vulnerable systems are not exposed to the network stack, and the read/write/execute capabilities of operating systems usually are required to launch an attack.

Attack Complexity (AC) describes the conditions beyond the attacker's control that must exist for exploits. Such conditions may require more information collection about the target system, specific system configuration settings, or execution exceptions. As shown in Figure 2(b), about 89.3% (369/413) vulnerabilities can be exploited with low attack complexity. It suggests that many vulnerabilities impose no restrictions on attackers, and any attackers are capable of exploiting the vulnerabilities without much effort and expertise.

Finding 1: About two-third (67.8%) vulnerabilities can be exploited via the network, and a significant amount (89.3%) of vulnerabilities can be exploited with low efforts.

2) *Analysis of PoC exploits:* By analyzing the collected PoC exploits, we find that a typical exploit can be divided into following steps:

- **Getting access to devices/systems.** We observe that the details of this step usually are missing in these PoC exploits. It mainly accounts for two reasons: (1) security researchers consider the malicious users and insiders as the most realistic threat; (2) log files are publicly accessible via web servers. In both cases, attackers do not need to spend efforts to get access to devices.
- **Retrieving log files from devices/systems.** The sensitive log files usually are generated by systems at runtime and stored

in certain directories on devices, and it is trivial for attackers to retrieve log files based on the PoC exploits. We find out two main situations increasing the attack complexity: (1) attackers need to brute force the actual paths of log files according to their naming conventions; (2) attackers need to perform specific operations to trigger the generation of log files, and sometimes user interaction is required.

- **Extracting sensitive information from log files.** There are only 4 out of 49 vulnerabilities where the sensitive information is not stored in cleartext. Moreover, the majority (40/49) of the PoC exploits provide the samples of sensitive log entries. By looking into the log entries, we find that it is straightforward for attackers to use keywords to locate sensitive information or devise regex expression to extract sensitive information.

In a nutshell, 10 out of 49 PoC exploits can be used by any attackers without the expertise, because attackers just need to visit specific URLs in browsers. If we take the users and insiders into consideration, the number of PoC exploits without requirements for expertise increases to 23. The malicious users and insiders just need to follow the instructions to execute some commands in Shell or perform some operations on GUI. As for the rest PoC exploits, attackers can not use them to launch attacks directly. One half requires attackers to have background knowledge to parse the log files and decode sensitive information. Another half comes to android applications, which usually require attackers to trick victims into installing malicious applications.

Finding 2: there are about half of (46.9% , 23/49) PoC exploits can be used by malicious users and insiders without any expertise to launch attacks.

B. RQ2: What are the root causes for the vulnerabilities?

Following the steps described in Section II-C, we categorize the root causes into 7 categories as shown in Table II. Note that there are three vulnerabilities whose root causes are unknown, because we do not collect sufficient resources to determine their root causes. In the context of data security, the data can be divided into three types according to their states: data-in-use, data-in-transit, and data-at-rest [6]. Each data state usually presents different security challenges as well as root causes and prevention solutions for data exposure. In our study, 107 vulnerabilities happen to the data-in-use state, which means the exposed data are being processed by systems, and 33 vulnerabilities occur at the data-at-rest state, which implies the exposed data are stored in persistent storage. Moreover, as Table II shows, the root causes of the vulnerabilities in these two data states are classified into five categories and two categories. In the rest of this section, we follow the order in Table II to describe these categories.

Insecure whole-object logging. Whole-object logging refers to that developers make logging calls with direct reference to composite objects. The underlying string conversion functions (e.g., `toString()` in Java and `__str__()` in Python) are

TABLE II: Root Causes of Vulnerabilities

State	Root Causes	Freq.
Data in use	Insecure whole-object logging	62
	Improper implementation of sanitization	16
	Unawareness of the logging in dependent systems	15
	Overlooking the risk of data exposure through logging	11
	Unanticipated use of insecure APIs to pass sensitive data	3
Data at rest	Incorrect permission assignment	25
	Improper preservation of permissions	8
	Unknown	3
	Total	143

called implicitly to convert the entire object into a single string. However, this coding practice is insecure, as it is difficult to recognize the sensitive data in these objects for developers, and hence they may not take action to prevent sensitive information exposure. On the one hand, the input domains of these objects can be complex and sometimes are derived from the user input. The actual data in these objects vary in different contexts. Looking into the sources of logged objects in these 62 vulnerabilities, we find that the top three common sources are HTTP request/response (23), configuration (20), and input/output of shell command (8). On the other hand, the structures of these objects can be dynamic, especially for the objects of collection, such as *map* in Java and *dict* in Python. Figure 3-a shows the partial patches for *CVE-2015-8749* of OpenStack. The vulnerable method attaches storage volumes to hypervisors with the parameter *connection_data*, which is a *dict* and contains configurations required to access storage volumes. Thus, developers log *connection_data* at the entrance of the method for problem diagnosis. However, *connection_data* occasionally contains credentials to access specific storage volumes.

Besides, the data in these objects may evolve along with the software. While developers have taken measures to prevent sensitive information exposure, it is possible to fail due to the introduction of new types of sensitive data. Figure 3-b shows the partial patches for *CVE-2017-2592* of OpenStack. The vulnerable method logs the HTTP request when exceptions occur in request handler. Developers have recognized the sensitive *HTTP_X_AUTH_TOKEN* in the HTTP request and mask it with asterisks. However, new types of tokens appear in the HTTP request as the software evolves, and existing sanitization can not handle them.

Developers introduce various sanitization functions (e.g., redaction and hashing) to prevent the information exposure. However, it is not trivial to implement sanitization functions. As Table II shows, **improper implementation of sanitization** results in 16 vulnerabilities. There are two main reasons:

- Developers make wrong assumptions about the structures of sensitive data. For example, in *CVE-2019-10195*, developers assume that the sensitive source is a one-level key-value map, so they simply iterate over the map to redact sensitive values at first level. However, the sensitive data can be a nested map, and developers fail to redact sensitive values in deeper levels.
- The configurations of sanitization functions are not correctly

```

class VolumeOps(object):
    def _connect_hypervisor_to_volume(self, sr_ref, connection_data):
        # connection_data can have credentials in it so make sure
        # to scrub those before logging.
        LOG.debug("Connect volume to hypervisor: %s",
                connection_data)
        strutils.mask_password(connection_data)

(a) CVE-2015-8749

+ _TOKEN_RE = re.compile('^X-\w+-Token:.*$', flags=re.MULTILINE)
class CatchErrors(base.ConfigurableMiddleware):
    def __call__(self, req):
        try:
            response = req.get_response(self.application)
        except Exception:
            if hasattr(req, 'environ') and 'HTTP_X_AUTH_TOKEN' in req.environ:
                req.environ['HTTP_X_AUTH_TOKEN'] = '*****'
            req_str = _TOKEN_RE.sub(r'\1: *****', req.as_text())
            LOG.exception(_LE('An error occurred during '
                'processing the request: %s'), req)
                'processing the request: %s'), req_str)

(b) CVE-2017-2592

```

Fig. 3: Partial Patches for Vulnerabilities

handled. To cope with complex input domains, software sometimes support users to customize sanitization functions via configurations. However, the defects in the configuration process can lead to malfunction of sanitization. For example, incorrect exception handling may lose the configurations of sanitization functions (*CVE-2019-14858*).

Unawareness of the logging in dependent systems. The functionality of software often depends on external systems. The unexpected logging in dependent systems account for 15 vulnerabilities in our data. Half of them happen to explicit dependent systems, which means the software explicitly invokes these systems. However, developers do not realize that they need to change the logging settings of these systems to meet their security needs. Another half of these vulnerabilities relate to implicit dependent systems, such as runtime environment. For instance, web applications usually run on web servers and are accessed via browsers by users. Sometimes, developers unaware that the URL parameters are archived by access log and browser history, and they send sensitive data with URL parameters when constructing web applications.

Overlooking the risk of data exposure through logging. There are 11 vulnerabilities whose actual arguments used in logging calls are obvious sensitive data. It is trivial to identify these sensitive data based on the names of the logged variables, such as *password* and *secret*. In particular, most (7/11) of these sensitive data is logged with high logging levels (e.g., INFO), which means they are likely to be exposed to log files in production mode. The developers should not make the obvious mistake if they are aware of the risk of data exposure through logging.

Unanticipated use of insecure APIs to pass sensitive data. As the title describes, developers sometimes do not realize that the parameters of specific APIs can flow into log files and use them to pass sensitive data. Actually, there are alternative secure APIs to pass sensitive data in these software. Therefore, the developers usually remove the insecure APIs or add warning message in documentation to fix these vulnerabilities.

Incorrect permission assignment means that the log files containing sensitive information are created with incorrect permission. There are two main causes for these 25 vulnerabilities: 1) The log files are stored under web-root. The web-root

directories of web servers are publicly accessible by default. 2) The log files are stored under temporary directories at the server, such as the *tmp* directory in Linux (CVE-2018-19960). Any logged-in users have access to these directories by default.

Improper preservation of permissions indicates that the permissions of log files are not correctly preserved when copying, restoring, or sharing, which can cause them to have less restrictive permissions than intended. For the 8 vulnerabilities fall into this category, the permissions of log files are compromised mainly due to log viewer (CVE-2019-0202) and log aggregation (CVE-2015-3243).

Finding 3: The top 3 common root causes for the vulnerabilities are insecure whole-object logging (62/143, 43.4%), incorrect permission assignment (25/143, 17.5%), and improper implementation of sanitization (16/143, 11.2%).

IV. DISCUSSION

A. Implications

We discuss the implications of our study from the perspectives of researchers, developers, and operators.

For researchers, we suggest that it is necessary to build more practical solutions to detect information exposure from logging. Previous solutions mainly depend on static/dynamic taint analysis [7], which requires users to set sources, sinks, and sanitization functions at first. However, it is often difficult to identify all of these functions, notably the source functions. Besides, sanitization functions sometimes are not implemented correctly. It is also necessary to propose approaches to detect the defects in sanitization functions.

For developers, they should pay attention to the vulnerabilities at the data-in-use state. During the software design phase, they should find out the sensitive data in the software as thorough as possible and design reliable sanitization functions to identify these sensitive data. During the software implementation phase, they should take care of the logging statements in software, especially for whole-object logging and obvious sensitive logging. Besides, they can build coding conventions and examine the logging statements during code review.

For operators, the vulnerabilities at the data-at-rest state are their primary concern. They need to assign correct permissions of log files and maintain the permissions over the lifetime of log files.

B. Threats to Validation

Internal Validity. The first threat to validity is that we answer the RQ2 based on manual analysis. To mitigate the bias when categorizing the vulnerabilities, we have two different individuals to perform the categorization independently, and the result will only be accepted when they reach consensus.

External Validity. The biggest threat to validity is the fact that our study relies on the vulnerabilities from NVD. Despite that NVD does not collect all real-world vulnerabilities, it is the most widely-used vulnerabilities database for research.

Besides, the collected vulnerabilities cover a wide variety of software projects of different sizes, domains, and programming languages. Therefore, the findings in our study may generalize to a much broader set of software projects.

V. RELATED WORK

The most relevant work to ours is a preliminary study performed by Zhou et al. [3] on sensitive information leakage through logging in Android applications. They proposed an approach to identify log statements in applications that expose sensitive information and found that there are 200 out of 5,000 applications leaking sensitive information through logging. However, our work differs from them in the following three ways: 1) we aim to understand vulnerabilities from various aspects rather than detecting vulnerabilities; 2) we focus on known, severe and exploitable vulnerabilities rather than those found by experimental detection tools; 3) we study more types of systems rather than merely Android applications.

VI. CONCLUSION

In this paper, we have presented the first attempt to investigate the vulnerabilities that expose sensitive information through logging. We have analyzed 413 real-world vulnerabilities in NVD, and 143 of them happen to open-source software projects. By analyzing the exploitability and root causes of these vulnerabilities, we reveal some interesting findings and discuss the implications for researchers, developers and operators. In the future, we intend to analyze the patches of these vulnerabilities and study how to assist developers to detect and fix these vulnerabilities.

ACKNOWLEDGMENT

This work was supported in part by the National Key Research and Development Program of China under Grant 2017YFB1400603, in part by the National Natural Science Foundation of China under Grant 61825205 and Grant 61772459, and in part by the National Science and Technology Major Project of China under Grant 50-D36B02-9002-16/19.

REFERENCES

- [1] "2019 CWE Top 25 Most Dangerous Software Errors," Accessed: 2020-05-01. [Online]. Available: <https://cwe.mitre.org/top25>
- [2] "Keeping Passwords Secure," Accessed: 2020-05-01. [Online]. Available: <https://about.fb.com/news/2019/03/keeping-passwords-secure/>
- [3] R. Zhou, M. Hamdaqa, H. Cai, and A. Hamou-Lhadj, "MobiLogLeak: a preliminary study on data leakage caused by poor logging practices," in Proceedings of 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering, 2020, pp. 577–581.
- [4] P. Johnson, R. Lagerström, M. Ekstedt, and U. Franke, "Can the Common Vulnerability Scoring System be Trusted? A Bayesian Analysis," IEEE Transactions on Dependable and Secure Computing, vol. 15, no. 6, pp. 1002–1015, 2018.
- [5] D. Spencer, Card sorting: Designing usable categories. Rosenfeld Media, 2009.
- [6] A. Shabtai, Y. Elovici, and L. Rokach, A Survey of Data Leakage Detection and Prevention Solutions. Springer Publishing Company, Incorporated, 2012.
- [7] J. Bell and G. Kaiser, "Phosphor: Illuminating Dynamic Data Flow in Commodity Jvms," in Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications, 2014, pp. 83–101.